Dependency Syntax

CS 485, Spring 2026
Applications of Natural Language Processing

Katrin Erk
College of Information and Computer Sciences
University of Massachusetts Amherst

With slides by Brendan O'Connor

Dependency structure

- Constituent structure groups words into constituents

  [NP I] saw [NP a fascinating painting] yesterday

  [NP She] [VP sneezed loudly]

- Dependency structure identifies links between words in the sentence
- These links have types such as subject, direct object

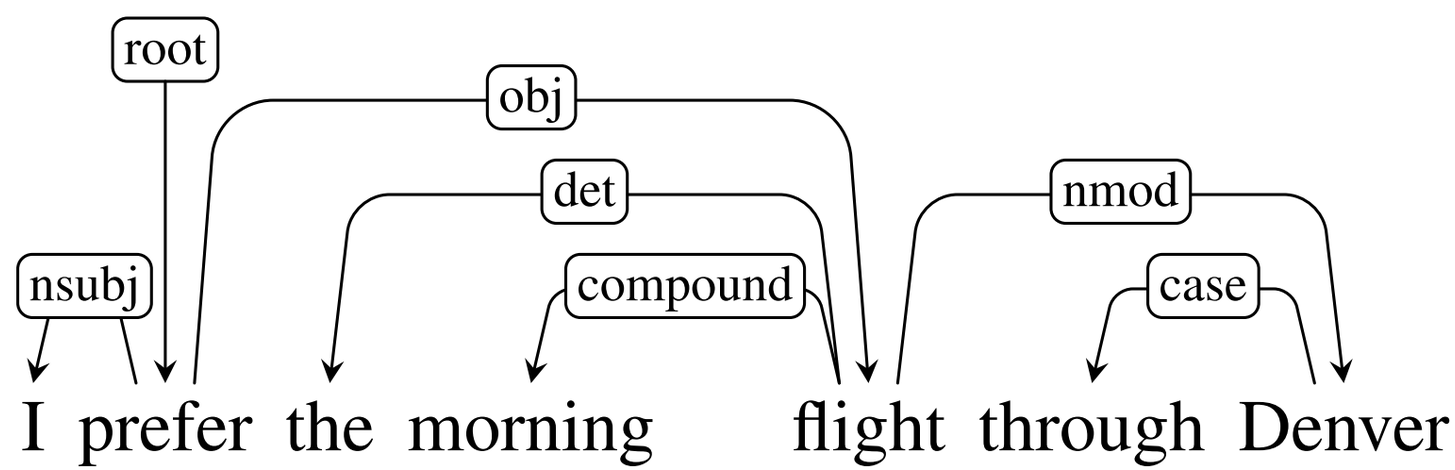
Dependency structure

- Constituent structure groups words into constituents

  [NP I] saw [NP a fascinating painting] yesterday

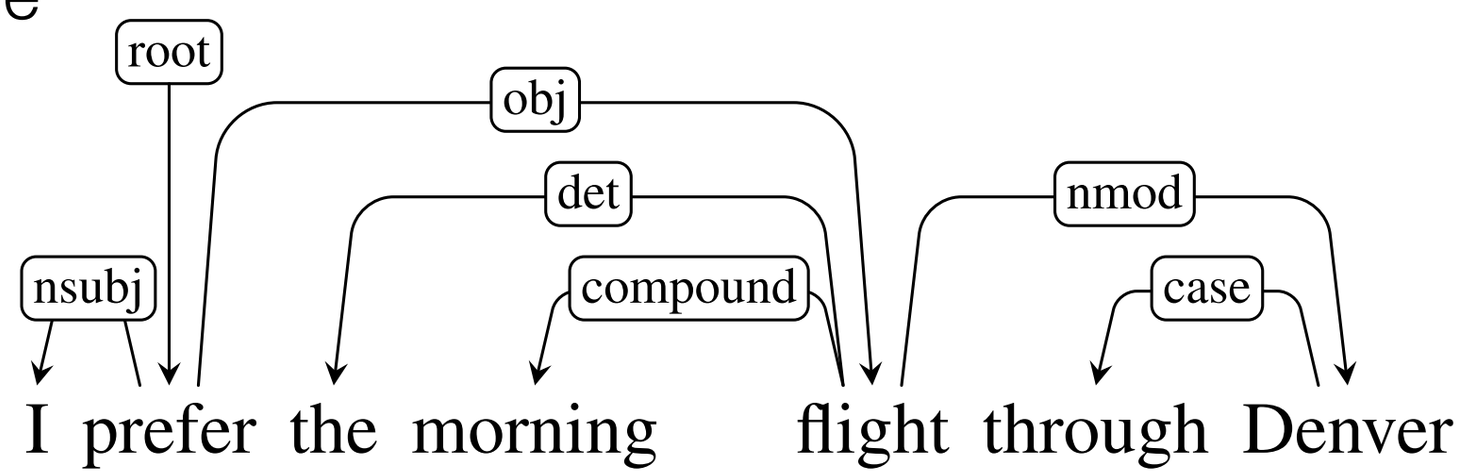
  [NP She] [VP sneezed loudly]

- Dependency structure identifies links between words in the sentence
- These links have types such as subject, direct object



Dependency structure: the example in more detail

- Directed acyclic graph (here, and often, a tree)
- Root: “prefer”
- “prefer” linked to “I”, link label “subj”:
  “I” is the subject of “prefer”
- “prefer” linked to “flight”, link label “obj”:
  “flight” is the object of “prefer”
- Determiner is the child node of its noun,
  as in flight —det—> the
- Preposition is the
  child node of its noun



More about dependency structure

- Labeled directed graph among all words tokens in a sentence
  - Every word has exactly one parent
  - Single root node. Often a tree
- Edge labels indicate grammatical relationships
- Dependency structures work well with free word order languages. (Why do constituent structures work less well?)

Clausal argument relations	Description
NSUBJ	Nominal subject
OBJ	Direct object
IOBJ	Indirect object
CCOMP	Clausal Complement
Other notable relations	**Description**
CONJ	Conjunct
CC	Coordinating conjunction

Nominal Modifier Relations	Description
NMOD	Nominal modifier
AMOD	Adjectival modifier
APPOS	Appositional modifier
DET	Determiner
CASE	Prepositions, postpositions, and other case markers

Some Universal Dependency relations

From constituency structure to dependency graphs

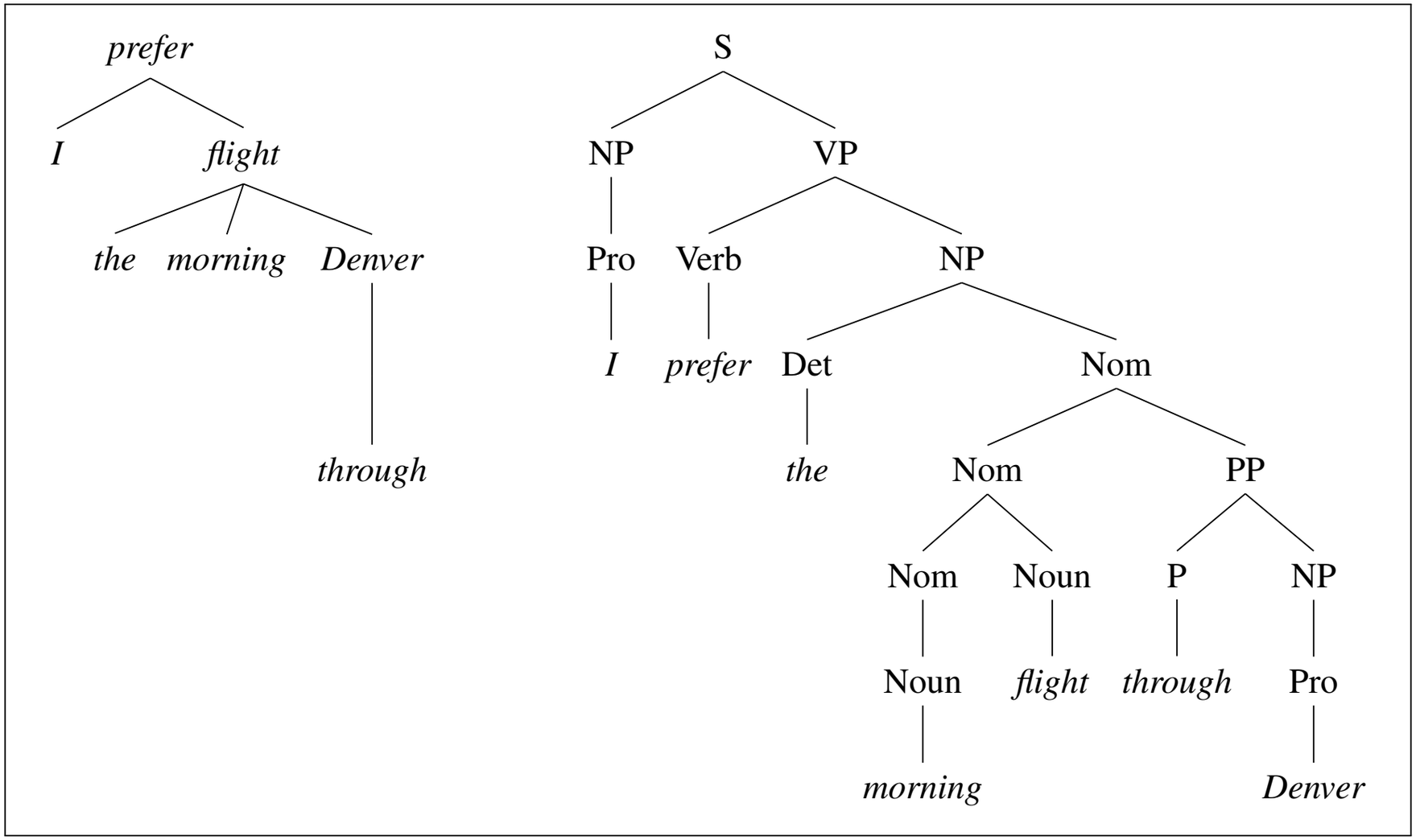
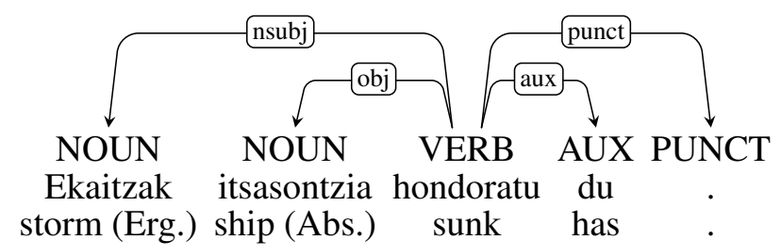


Figure 19.1 Dependency and constituent analyses for *I prefer the morning flight through Denver.*

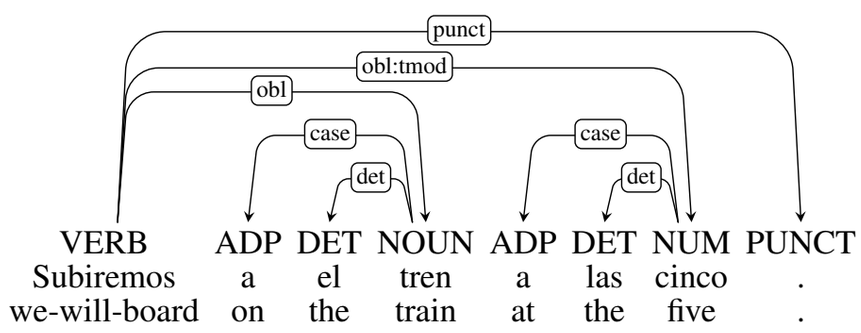
- Dependency subgraph often corresponds to a some constituent
- The de subgraph root is the "head" of that constitutent

Universal Dependencies

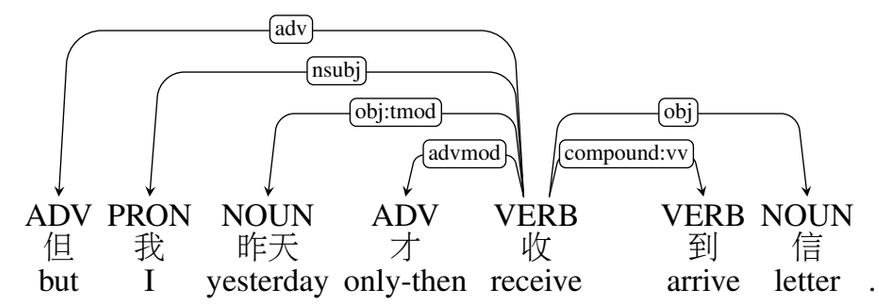
- Dependency treebanks are available for *many* different languages
  - https://universaldependencies.org/
- Many open-source dependency parsers (and tagging/POS/morphology) trained on them are also widely available; e.g. Stanza, SpaCy, etc.



[Basque] Ekaitzak itsasontzia hondoratu du. "The storm has sunk the ship."(19.5)



[Spanish] Subiremos al tren a las cinco. "We will be boarding the train at five."(19.4)



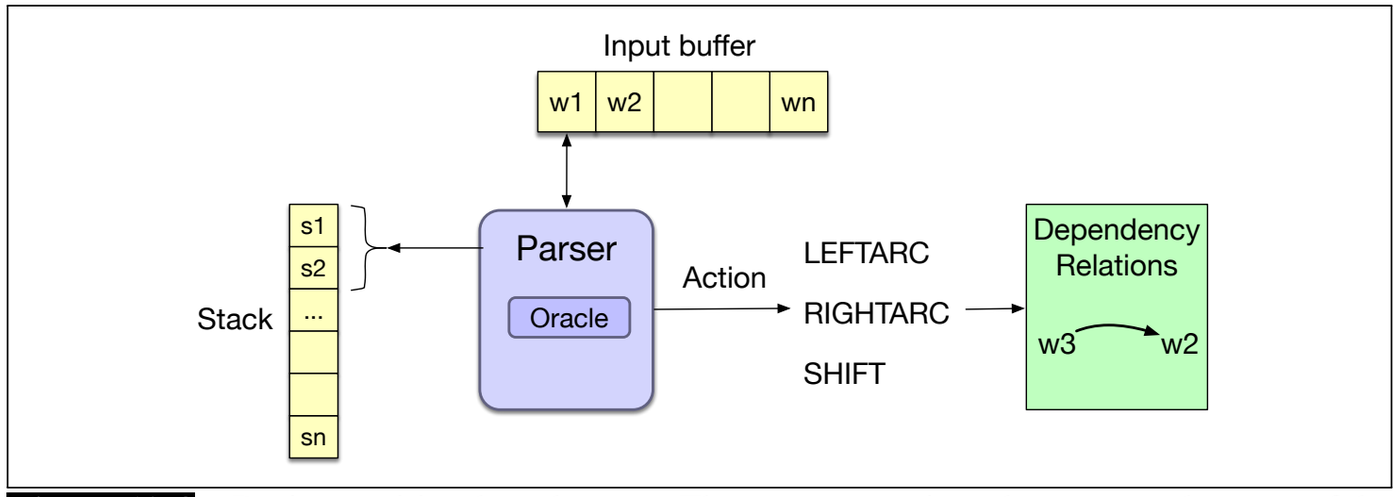
[Chinese] 但我昨天才收到信 "But I didn't receive the letter until yesterday."(19.6)

Shift-reduce, transition parsing

- How to predict a parse structure for an input sentence? Deal with massive ambiguity.
- *Incremental parsing*: proceed left-to-right, building up the parse structure incrementally.
  - Interesting analogy for human sentence processing
  - (Many similar algorithms exist for both constituency and dependency parsing)

Transition-based parsing

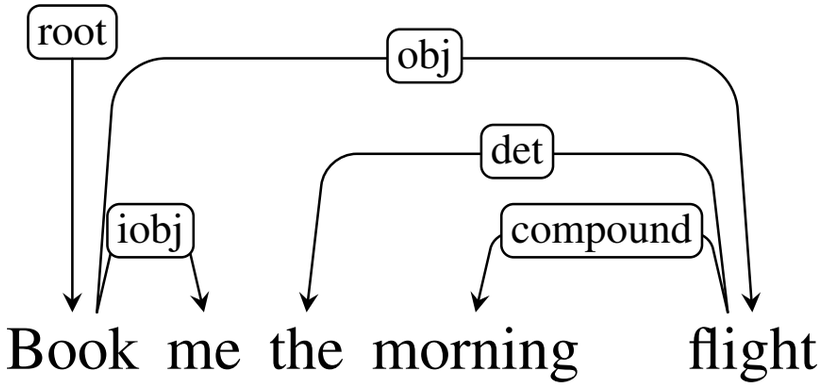
- Parser reads from the input, left to right
- Stack on which it can shift not-yet-analyzed tokens
- State machine with just 3 allowed actions:
  - shift (shift to stack),
  - left-arc among the two top members of the stack
  - right-arc among the top two members of the stack
- Machine learning based classifier ("Oracle" in the picture below) decides on action



from JM3

Figure 19.4 Basic transition-based parser. The parser examines the top two elements of the stack and selects an action by consulting an oracle that examines the current configuration.

Parse example (from JM)



Step 2, after the word me has been pushed onto the stack

Stack	Word list	Relations
[root, book, me]	[the, morning, flight]	

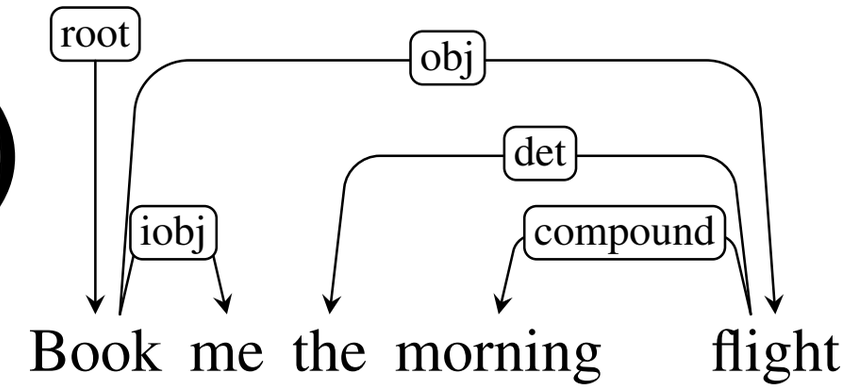
Right-arc: "book" is head of "me", pop "me" off the stack

Stack	Word list	Relations
[root, book]	[the, morning, flight]	(book -> me)

Then several shifts

Stack	Word list	Relations
[root, book, the, morning, flight]	[]	(book -> me)

Parse example (from JM)



Stack	Word list	Relations
[root, book, the, morning, flight]	[]	(book -> me)

All words have been passed on the stack. Remaining steps: Reduce Left-arc, and remove the child of the new arc from the stack

Stack	Word list	Relations
[root, book, the, flight]	[]	(book -> me), (morning <- flight)

Note: There are often multiple action sequence that lead to the right parse.
Success rests on getting the right action predictions.

Learning to predict the action

- Feature templates: can use all information from current parser state (stack, buffer, edges so far)
  - Current word on top of stack, buffer, edges? At second position?
  - Current POS tag on top of stack, buffer, edges? At second position?
  - Are they left or right of each other in the sentence?
  - etc.
- Training time: use rule system to extract oracle transition from gold-standard annotations

Dependency applications

- Dependency paths (e.g. *(fly, -nsubj->, bird)*) can be used as less sparse alternative to n-grams
  - Sometimes helps, sometimes doesn't
- Dependency relations can be selected for semantic relationships
- At a higher level, word-to-word dependencies are key to current "Transformer" neural net models, but explicit syntactic dependencies are used less often